Open Access | https://doi.org/10.25080/eykk7863



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Density Functions and Random Number Generators of Alpha-Stable Distributions

Wael Tabbara[1], **Sharafeddine Sharafeddine**[2,1], **Ahmad El-Hajj**[1], **Jihad Fahs**[1], and **Ibrahim Abou-Faycal**[1]

[1]Electrical & Computer Eng. Department, American University of Beirut, Lebanon, [2]School of Electrical & Computer Engineering, Cornell University, USA

Abstract

Heavy-tailed distributions are increasingly found to better fit empirical data in engineering, finance, physics, network science, and related fields. Among them, alpha-stable distributions play a central role being limiting laws in the generalized central limit theorem: they are expected to be exceptionally good models whenever sums of multiple independent heavy-tailed sources are at play. Despite their theoretical importance, their practical use remains challenging: alpha-stable probability densities generally do not have closed-form expressions, and numerical evaluation and random variate generation can be difficult, especially in the multivariate setting. This paper presents the American University of Beirut Heavy Tails Package (AUB-HTP), a Python package for numerical computation and simulation of alpha-stable distributions. The package provides scalar density evaluation using several complementary methods, including Zolotarev-type integral representations, series formulas, and numerical inversion of characteristic functions. It also provides random variate generation for scalar and multivariate alpha-stable distributions, with support for flexible spectral measures through LePage series representations. Numerical experiments demonstrate that AUB-HTP improves the accuracy, stability, and parameter coverage of existing tools for scalar density computation, while adding new capabilities for multivariate simulation. The package is designed to support reproducible computational work involving heavy-tailed models across a broad range of scientific applications.

Keywords Heavy Tailed Data, Heavy Tails, Alpha Stable, Levy Stable

Published Jul 13, 2026

Correspondence to
Ahmad El-Hajj
aae108@mail.aub.edu

Open Access

Copyright © 2026 Tabbara *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Heavy-tailed distributions arise naturally in many scientific and engineering applications, including communications, signal processing, finance, physics, and network modeling [1], [2], [3], [4], [5]. In such settings, rare but large deviations occur more frequently than predicted by light-tailed models such as the Gaussian distribution. As a result, Gaussian-based modeling can significantly underestimate the probability and impact of extreme events.

Among heavy-tailed models, α-stable distributions occupy a central theoretical position. They generalize the Gaussian distribution through the stability property: sums of independent α-stable random variables remain α-stable, up to location and scale changes. They also arise as limiting distributions for normalized sums of independent random variables under the generalized central limit theorem [6]. These properties make α-stable laws natural candidates for modeling aggregate effects generated by many independent heavy-tailed sources.

A major practical limitation of α-stable distributions is that their probability density functions (PDFs) generally do not admit closed-form expressions, except in the special cases of

the Cauchy and Lévy distributions. Consequently, numerical tools are required for density evaluation, modeling, and simulation. Existing approaches for scalar α-stable random variate generation, such as the Chambers–Mallows–Stuck (CMS) method [7], are well established, but they do not directly address general multivariate α-stable distributions. In the multivariate case, available simulation methods typically focus on special spectral measures, such as isotropic or sub-Gaussian models [4], rather than arbitrary spectral measures.

Available numerical packages for α-stable distributions include J. Royuela-del Val, F. Simmross-Wattenberg, and C. Alberola-López [8] that provides an R and C++ implementation for α-stable random variable generation using a modified version of the CMS method [7], [9]. In M. Teimouri, M. Torshizi, A. Mohammadpour, and S. Nadarajah [10] the authors provide an R package that supports generating an α-stable random variable using the same modified CMS method; a random variable from a truncated α-stable distribution; an α-stable random vector with a sub-Gaussian spectral measure, and other bivariate α-stable random vectors. The work in B. J. Swihart and J. P. Nolan [11] presents another α-stable package which references M. Teimouri, M. Torshizi, A. Mohammadpour, and S. Nadarajah [10] as well. When it comes to density computations, the `libstable` package [8] is based on Zolotarev's integral representation [12] computed using Gauss-Kronrod quadrature over the entire parameter space. The package does the evaluations in C/C++ and provides a MATLAB front-end in addition to an R package. The `alphastable` R package [10] evaluates the density using Skorohod-type asymptotic series [13], [14], [15] wherever those series converge. The package falls back to `stabledist` [16] elsewhere, which computes the density via Zolotarev's integral representation [12] using adaptive quadrature.

This paper presents the American University of Beirut Heavy Tails Package (AUB-HTP)[1], a Python package for computational work with α-stable distributions. The package provides scalar density computation using several numerical methods, including Zolotarev integral representations [12], Skorohod-Pollard-Bergström formulas [13], [14], [15], and direct numerical inversion of the *characteristic function*[2]. These methods provide complementary tradeoffs in terms of accuracy, speed, and robustness across parameter regimes. In addition, AUB-HTP implements random variate generation for scalar and multivariate α-stable distributions. For the multivariate case, the package uses LePage series representations [17], which support flexible spectral measures and perform particularly well in challenging heavy-tailed regimes, including small values of α.

The goal of AUB-HTP is to make α-stable modeling more accessible to computational scientists by providing reliable density evaluation, flexible simulation tools, and reproducible numerical routines within the Python ecosystem. The main contributions of this work are: (i) a unified Python implementation of several scalar α-stable PDF evaluation methods; (ii) a flexible random variate generator for scalar and multivariate α-stable distributions; (iii) support for general spectral measures through LePage series; and (iv) numerical benchmarks comparing accuracy, runtime, and robustness against existing implementations.

Notations

We adopt the following conventions. The vector $\boldsymbol{x} \in \mathbb{R}^d$ represents a column vector and $\boldsymbol{x}^T$ is its transpose. The Euclidean inner product of two vectors $\boldsymbol{x} = (x_1, x_2, \cdots, x_d)^T$ and $\boldsymbol{y} = (y_1, y_2, \cdots, y_d)^T$ is denoted by $\boldsymbol{x}^T\boldsymbol{y} = \langle \boldsymbol{x}, \boldsymbol{y} \rangle = \sum_{i=1}^{d} x_i y_i$ and the L_2 norm of $\boldsymbol{x}$ by $\parallel \boldsymbol{x} \parallel = \sqrt{\langle \boldsymbol{x}, \boldsymbol{x} \rangle}$. The unit sphere in $\mathbb{R}^d$ is denoted by $\mathbb{S}^{d-1} = \{\boldsymbol{x} \in \mathbb{R}^d : \parallel \boldsymbol{x} \parallel = 1\}$. The class of all Borel subsets in a metric space S is represented by $\mathcal{B}(S)$. We use $\boldsymbol{X} \perp\!\!\!\perp \boldsymbol{Y}$ to indicate that the two random vectors $\boldsymbol{X}$ and $\boldsymbol{Y}$ are independent. When a random vector $\boldsymbol{X}$ is distributed

[1] Source repository: https://github.com/AUB-HTP/AUB-HTP. Archived releases: https://pypi.org/project/aub-htp/.

[2] The characteristic function of a random variable X is given by the expected value of e^{itX}, $\phi_X(u) = \mathbb{E}[e^{iuX}]$. It uniquely determines the distribution of X, and it always exists.

uniformly on the unit sphere, we write $\boldsymbol{X} \sim \mathcal{U}(\mathbb{S}^{d-1})$. For a random variable X that is Gamma distributed, we write $X \sim \mathrm{Gamma}(\alpha, \beta)$, and we use the following parameterization for the PDF

$$f_X(x) = \begin{cases} \frac{1}{\Gamma(\alpha)\beta^\alpha} x^{\alpha-1} e^{-\frac{x}{\beta}} & \text{if } x > 0 \\ 0 & \text{otherwise,} \end{cases}$$

where $\Gamma(\cdot)$ is the Gamma function.

2. Preliminaries on α-Stable Distributions

For a definition of multivariate stable vectors, we refer the reader to G. Samorodnitsky and M. S. Taqqu [18, Definition 2.1.1]. In particular, we are interested in α-stable [3] random vectors.

Definition 2.1.

[18]

Let $0 < \alpha < 2$, then $\boldsymbol{X} = (X_1, X_2, \cdots, X_d)$ is an α-stable random vector in $\mathbb{R}^d$ if and only if there exists a finite measure Λ on $\mathbb{S}^{d-1}$ called the "spectral" measure and a vector $\boldsymbol{\mu}^0$ in $\mathbb{R}^d$ such that the characteristic function of $\boldsymbol{X}$ is given by

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = \mathbb{E}[\exp\{\mathrm{i}\langle \boldsymbol{t}, \boldsymbol{X}\rangle\}] = \exp\left\{ -\int_{\mathbb{S}^{d-1}} \psi_\alpha(\langle \boldsymbol{t}, \boldsymbol{s}\rangle)\, \Lambda(d\boldsymbol{s}) + \mathrm{i}\langle \boldsymbol{t}, \boldsymbol{\mu}^0\rangle \right\}, \tag{1}$$

and where

$$\psi_\alpha(u) = \begin{cases} |u|^\alpha \left[1 - \mathrm{i}\,\mathrm{sign}(u) \tan\frac{\pi\alpha}{2}\right] \alpha \neq 1 \\ |u| \left[1 + \mathrm{i}\frac{2}{\pi}\,\mathrm{sign}(u) \ln|u|\right] \quad \alpha = 1. \end{cases}$$

In the scalar case ($d = 1$) –and whenever $0 < \alpha < 2$, the spectral measure boils down to two mass points on $s = \pm 1$, and the characteristic function in (1) is expressible in terms of 4 parameters: the stability index $0 < \alpha < 2$, skewness $-1 \leq \beta \leq 1$, scale $\gamma > 0$, and location $\delta \in \mathbb{R}$. In this case, we denote an α-stable random variable X using the same notation as in Nolan [9], $X \sim \mathcal{S}(\alpha, \beta, \gamma, \delta; k)$ where k is the parameterization index (or type). We will use the symbols $\mathcal{S}_0$ and $\mathcal{S}_1$ to refer to the two common type-0 and type-1 parameterizations, respectively. In the multivariate case, we denote a d-dimensional non-degenerate α-stable random vector $\boldsymbol{X}$ with stability index α, spectral measure Λ and shift vector $\boldsymbol{\mu}^0$, by $\boldsymbol{X} \sim \boldsymbol{S}_\alpha(\Lambda, d, \boldsymbol{\mu}^0)$. Note that Definition 2.1 corresponds to the generalization of the scalar case under the parameterization $k = 1$.

Parameterizations of scalar α-stable distributions

The two parameterizations, 0 and 1 are denoted by $\mathcal{S}(\alpha, \beta, \gamma, \delta; 0)$ and $\mathcal{S}(\alpha, \beta, \gamma, \delta; 1)$ respectively and differ primarily in how they define the location (shift) of the distribution, particularly near $\alpha = 1$. For a given α and β, let Z be a "standard" α-stable variable corresponding to $\gamma = 1$ and $\delta = 0$.

Table 1. *Parametrization 0 and Parametrization 1.*

$\boldsymbol{X} \sim \mathcal{S}(\alpha, \beta, \gamma, \delta; 0)$	**Characteristic Function:** $\mathbb{E}[e^{i\boldsymbol{u}\boldsymbol{X}}]$	$\boldsymbol{\alpha}$
$X \overset{d}{=} \gamma(Z - \beta\tan(\frac{\pi\alpha}{2})) + \delta$	$\exp(-\mid \gamma u \mid^\alpha \left[1 + \mathrm{i}\beta\tan(\frac{\pi\alpha}{2})\,\mathrm{sign}(u)(\mid \gamma u \mid^{1-\alpha} - 1)\right] + \mathrm{i}\delta u)$	$\alpha \neq 1$
$X \overset{d}{=} \gamma Z + \delta$	$\exp(-\mid \gamma u \mid \left[1 + \mathrm{i}\beta\frac{2}{\pi}\,\mathrm{sign}(u)\log\mid \gamma u \mid\right] + \mathrm{i}\delta u)$	$\alpha = 1$

[3] The term α-stable is used to refer to the stable family excluding the Gaussian law.

$X \sim \mathcal{S}(\alpha, \beta, \gamma, \delta; 1)$	**Characteristic Function:** $\mathbb{E}[e^{iuX}]$	α
$X \stackrel{d}{=} \gamma Z + \delta$	$\exp(-\mid \gamma u \mid^{\alpha} [1 - \mathrm{i}\beta \tan(\frac{\pi\alpha}{2}) \operatorname{sign}(u)] + \mathrm{i}\delta u)$	$\alpha \neq 1$
$X \stackrel{d}{=} \gamma Z + (\delta + \beta\frac{2}{\pi}\gamma \log \gamma)$	$\exp(-\mid \gamma u \mid [1 + \mathrm{i}\beta\frac{2}{\pi} \operatorname{sign}(u) \log\mid u \mid] + \mathrm{i}\delta u)$	$\alpha = 1$

Parameterization 0 has the advantage of avoiding discontinuities near $\alpha = 1$ by adjusting the shift behavior and is often preferred for numerical stability [9]. On the other hand, explicit adjustment to the location is done when $\alpha = 1$ for parameterization 1.

Note that the choice of parameterization directly affects the shift and symmetry of the distribution, especially when $\beta \neq 0$ and α is close to 1. In both forms, the distribution can be standardized by setting $\gamma = 1$ and $\delta = 0$, in which case we use the abbreviations $\mathcal{S}(\alpha, \beta; 0)$ and $\mathcal{S}(\alpha, \beta; 1)$ respectively.

In the univariate case, our package can simulate both parameterizations $k = 0, 1$.

3. Scalar Density Evaluation

The AUB-HTP package provides numerical evaluation of scalar α-stable probability density functions through a SciPy-like interface. This component complements the random number generator: the generator produces samples, while the density module provides the reference curve used for model verification, visualization, and numerical comparison.

Except for Cauchy and Levy laws, α-stable distributions do not have a closed-form density, which must therefore be computed using numerical inversion or analytic representations that are valid in specific parameter regions.

Towards fulfilling our main objective of reducing expensive quadrature calls while preserving accuracy across a wide range of α, β, and sample values, the density evaluator combines three methods:

- direct inversion of Nolan's $\mathcal{S}_1$ characteristic function near the mode [9]
- Zolotarev's integral representation in the middle range [12]
- Skorohod-Pollard-Bergström's series expansions in the tails [13], [14], [19].

3.1. Input standardization

All density calls are first mapped to a standardized variable allowing the internal routines to work on standardized densities while still supporting user-facing location and scale parameters. More specifically, given an input sample value x the code evaluates

$$z = \frac{x - \text{shift}}{\gamma}, \tag{2}$$

then rescales the final density by $1/\gamma$. For the $\mathcal{S}_1$ parameterization for example, the shift is given by J. P. Nolan [9, equation 1.5]

$$\text{shift} = \begin{cases} \delta, & \alpha \neq 1 \\ \delta + \beta\frac{2}{\pi}\gamma \log \gamma & \alpha = 1, \end{cases}$$

as can be deduced from Table 1.

Additionally, if $f_{\alpha,\beta}(x)$ is the PDF of a normalized α-stable random variable, for negative evaluation points the code uses the symmetry relation J. P. Nolan [9, proposition 1.1]

$$f_{\alpha,\beta}(-x) = f_{\alpha,-\beta}(x),$$

and most formulas only need to be evaluated on the positive half-line.

3.2. Hybrid evaluation strategy

The main density function acts as a dispatcher. It divides the input array into regions and applies the method that is most accurate for each region.

- For small $|\ z\ |$ (see equation (2)), the package uses direct inversion of the characteristic function. While this method is slower than the series expansions, it is stable near the mode where tail formulas are not accurate.
- For intermediate values of $|\ z\ |$, the package uses Zolotarev's integral representation. This provides a useful compromise between direct inversion and asymptotic series methods.
- For large $|\ z\ |$, the package uses Skorohod-Pollard-Bergström series expansions. These formulas are fast in the tails and avoid repeated numerical integration.

The switching points between methods are not fixed constants. They are instead selected from precomputed cutoff functions depending on (α, β). These cutoffs were obtained using grid-based comparison with `scipy.stats.levy_stable.pdf`. At runtime, the code queries the corresponding cutoff and applies Boolean masks to route each input point to the selected method.

Density formulas

• For direct inversion in the $\mathcal{S}_1$ parameterization [9], the density is computed from the characteristic function. The standardized density is

$$f_{\alpha,\beta}(x) = \frac{1}{\pi}\int_0^\infty e^{-t^\alpha}\cos\Big(xt - \beta\tan\Big(\frac{\pi\alpha}{2}\Big)t^\alpha\Big)\,dt \quad \alpha \neq 1$$

$$f_{1,\beta}(x) = \frac{1}{\pi}\int_0^\infty e^{-t}\cos\Big(xt + \frac{2}{\pi}\beta t\log t\Big)\,dt \qquad \alpha = 1,\ \text{with the logarithmic correction.}$$

These integrals are evaluated with `scipy.integrate.quad_vec`.

• For the middle region, the code uses Zolotarev's integral representation [12]. For $\alpha \neq 1$ and $x > 0$,

$$f_{\alpha,\beta}(x) = \frac{\alpha x^{\frac{1}{\alpha-1}}}{\pi\ |\ \alpha-1\ |}\int_{-\theta_0}^{\pi/2} V(\theta)\exp\big[-x^{\frac{\alpha}{\alpha-1}}V(\theta)\big]\,d\theta,$$

where

$$-\theta_0 = \frac{1}{\alpha}\arctan\Big(\beta\tan\Big(\frac{\pi\alpha}{2}\Big)\Big),$$

and where $V(\theta)$ is evaluated following Zolotarev's formula [12]. This integral is computed using the function `scipy.integrate.quad`.

• For the tail region, we use Skorohod-type series expansions [13], [14], [19] of the form

$$f_{\alpha,\beta}(x) \approx \frac{1}{\pi x}\sum_{n=1}^{N} a_n x^{-\alpha n},$$

where

$$a_n = \frac{(-1)^{n-1}\Gamma(n\alpha+1)}{n!}\Big(1+\beta^2\tan^2\Big(\frac{\pi\alpha}{2}\Big)\Big)^{n/2}\sin\Big[n\Big(-\frac{\pi\alpha}{2} + \arctan\Big(\beta\tan\Big(\frac{\pi\alpha}{2}\Big)\Big)\Big)\Big].$$

The number N of terms used to calculate the series is appropriately chosen depending on the parameter range. In the current package, the $\alpha < 1$ branch uses a larger truncation level, while the $\alpha > 1$ branch requires fewer terms in the tested regimes. In some cases, minor spikes occurred at disjoint points away from the mode due to numerical irregularities. These spikes were detected by checking locations where monotonicity is violated. This is

resolved by applying –after the density is computed, a smoothing step enforcing monotonicity of $f_{\alpha,\beta}$ on either side of the mode.

3.3. Validation

The density evaluator was tested against `scipy.stats.levy_stable.pdf` on grids of α, β, and x. The near-mode tests used $x \in [-10, 10]$, while the tail tests examined values up to $x = 100$.

The metrics listed in Table 2 were used to compare the AUB-HTP PDF generator with that of SciPy.

Table 2. *Error and performance metrics comparing $\hat{f}$ against the SciPy reference f_{SciPy} over $\{x_i\}_{i=1}^{n}$, with $\varepsilon > 0$ guarding against division by zero.*

Metric	**Formula**
Maximum absolute error	$\max_{1\le i\le n} \mid \hat{f}(x_i) - f_{\text{SciPy}}(x_i) \mid$
Maximum relative error	$\max_{1\le i\le n} \frac{\mid \hat{f}(x_i) - f_{\text{SciPy}}(x_i) \mid}{\max(\mid f_{\text{SciPy}}(x_i) \mid, \varepsilon)}$
Mean absolute error	$\frac{1}{n}\sum_{i=1}^{n} \mid \hat{f}(x_i) - f_{\text{SciPy}}(x_i) \mid$
Runtime speedup factor against SciPy	$\frac{T_{\text{SciPy}}}{T_{\text{AUB-HTP}}}$

The tests show close agreement across most of the parameter space, but they also reveal regions where the SciPy reference exhibits irregular numerical behavior. The strongest agreement occurs away from the narrow region around $\alpha = 1$, where changes in the parameterization and the presence of logarithmic terms make density evaluation more sensitive. In this region, visual diagnostics are important because the relative difference may become large when the reference density is close to zero or numerically unstable. Figure 1 illustrates this behavior near $\alpha = 1$ using the maximum relative difference measured around the mode. Away from $\alpha = 1$, Figure 1 shows agreement with the SciPy reference. Results for the remaining error metrics, parameter ranges, and tail evaluations, together with the complete benchmark data, are available in the Github repository[4].

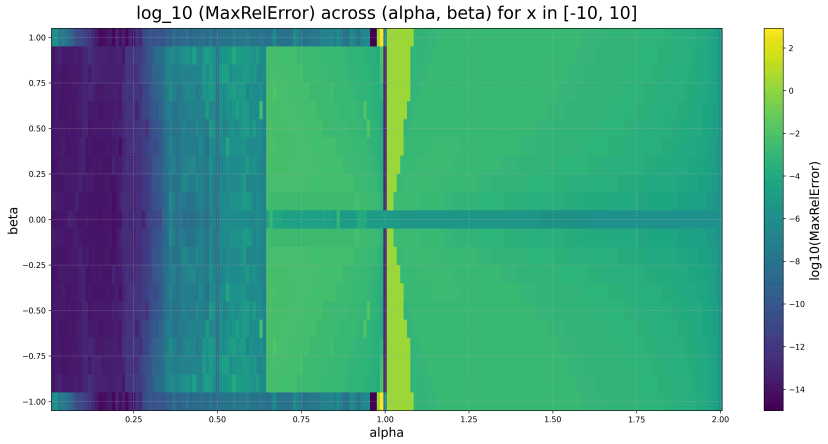


Figure 1. *Maximum relative difference across (α, β) near the mode. Larger values near $\alpha = 1$ mark the most sensitive numerical region.*

3.4. Discrepancies in the SciPy reference

While `scipy.stats.levy_stable.pdf` [20] is used as the main numerical reference, we observed that it has some discrepancies for certain parameter values. These discrepancies

[4] https://github.com/AUB-HTP/AUB-HTP/tree/main/benchmarks

appear mostly near the sensitive region around $\alpha = 1$, and they can create misleading relative difference values.

The main issue is that the SciPy PDF sometimes shows unusual zero or near-zero regions in some places while the densities are known to be smooth. Since the maximum relative difference contains a division by the reference value, even a small disagreement in such regions can produce a very large relative difference. Therefore, some of the large relative differences in the benchmark tables do not necessarily mean that our PDF generator is inaccurate. Instead, they are likely due to the instability or discontinuity in the reference PDF.

To validate this, we generated visual diagnostic plots comparing the AUB-HTP with the `levy_stable` PDF values. Figure 2 shows several cases where the SciPy reference generates unexpected zero windows or sharp changes, while the PDF generated using our package remains smooth.

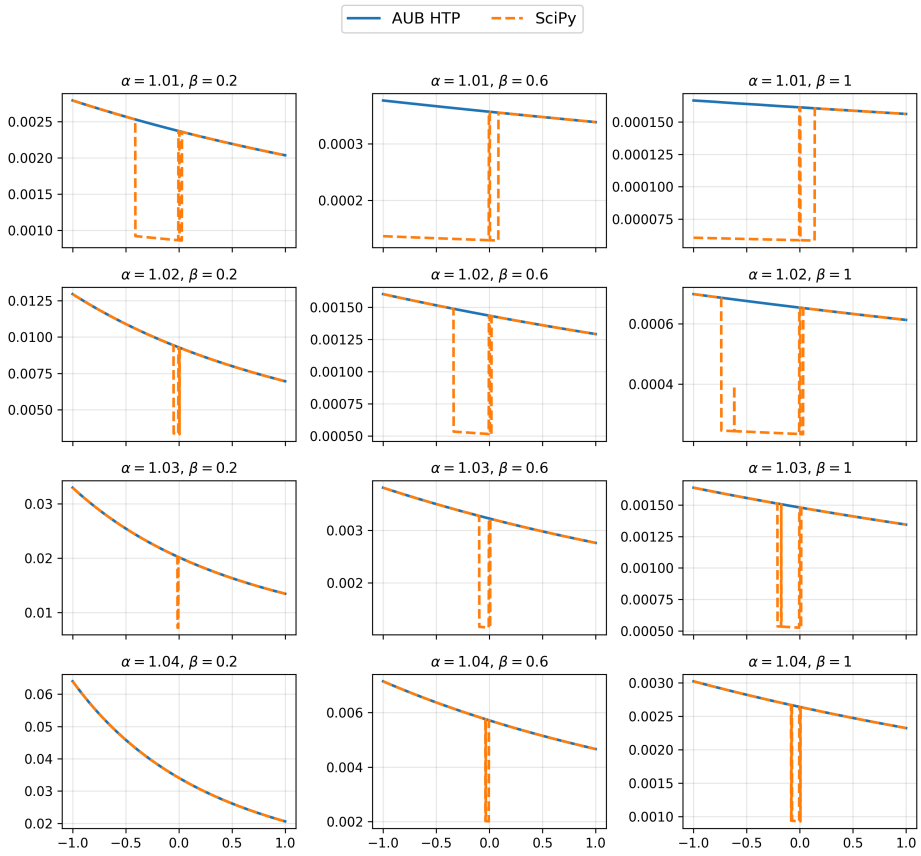


Figure 2. *Visual comparison between AUB-HTP PDF generator and `scipy.stats.levy_stable.pdf`. The plots zoom into regions where the SciPy reference becomes numerically zero or behaves irregularly. These regions explain why some relative difference values become large even when our generated density remains smooth.*

The heatmaps are useful for locating sensitive parameter regions, but the diagnostic plots were needed to understand whether the difference is primarily attributed to our method or the numerical reference.

3.5. Runtime

The adopted hybrid strategy reduces runtime by avoiding direct quadrature when a faster formula is accurate enough. The largest gains occur in the tail region, where series expansions replace repeated numerical integration.

In the benchmark runs, the median speedup was approximately $58\times$, with larger speedups in tail-heavy evaluations. These gains come from three design choices:

- vectorized array operations
- region-specific method dispatch
- reuse of precomputed cutoff functions.

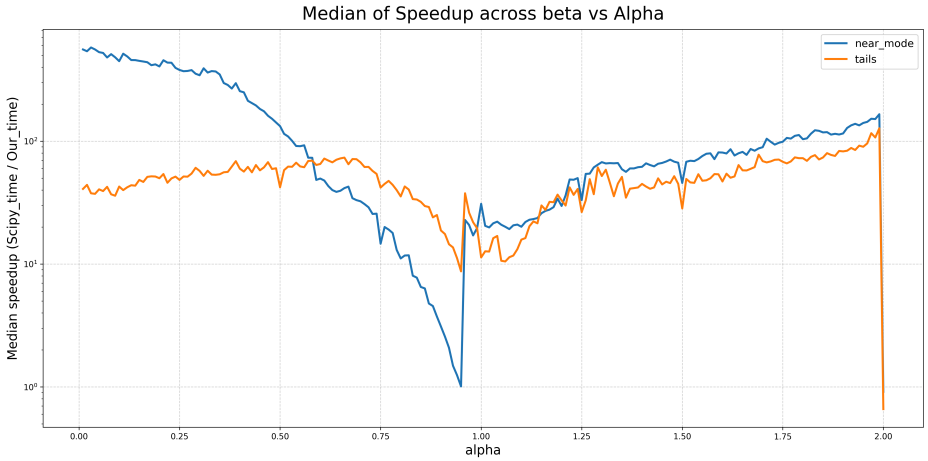


Figure 3. *Median runtime speedup relative to* `scipy.stats.levy_stable.pdf`*. The largest gains occur where the series expansions are selected.*

The speedup plot in Figure 3 shows that our method is strictly faster than `scipy.stats.levy_stable.pdf` for almost all tested values of α. Note that the vertical axis is on a logarithmic scale and values between 10 and 100 represent large runtime improvements. For small values of α, especially in the near-mode range, the speedup is very high. This is because the method selection avoids unnecessary numerical integration and uses the formulas that are faster in those regions. In the tail range, the speedup is also consistent for most values of α, since the Skorohod-Pollard-Bergström formulas replace repeated quadrature calls. The smallest speedups occur near $\alpha = 1$ which is expected since this region is numerically delicate. The formulas change around $\alpha = 1$, and the characteristic function contains a logarithmic correction term. The package therefore relies more on direct inversion in this region, which is slower than the series-based methods.

4. Generating α-Stable Random Vectors

In the univariate case, the package implements a numerically stable version of the Chambers, Mallows, and Stuck method which is presented in J. P. Nolan [9], R. Weron [21], and it is the same method as in J. Royuela-del Val, F. Simmross-Wattenberg, and C. Alberola-López [8], M. Teimouri, M. Torshizi, A. Mohammadpour, and S. Nadarajah [10]. This method is commonly considered to be the fastest and most accurate [22]. In the multivariate setup, the proposed random number generator is based on a theoretical foundation that we state next.

4.1. The LePage series

The LePage series [18] was first introduced in R. LePage, M. Woodroofe, and J. Zinn [17], and it was suggested as a simulation method in R. Weron [22]. This series allows for simulating a

multivariate stable vector with a stability index α and a given spectral measure through the use of the following theorem V. Bentkus, A. Juozulynas, and V. Paulauskas [23, Theorem 4].

Theorem 1.

Let $\boldsymbol{V_1}, \boldsymbol{V_2}, \cdots$ be independent and identically distributed (i.i.d) random vectors in $\mathbb{R}^d$ that are drawn according to a spectral measure Λ where $\Lambda(\mathbb{S}^{d-1}) > 0$, i.e.,

$$\Pr(\boldsymbol{V} \in A) = \frac{\Lambda(\mathbb{S}^{d-1} \cap A)}{\Lambda(\mathbb{S}^{d-1})} \quad \forall\ A \in \mathcal{B}(\mathbb{R}^d).$$

Let $E_1, E_2, \cdots$ be i.i.d. exponential random variables with mean 1, independent of $\boldsymbol{V}_1, \boldsymbol{V}_2, \cdots$, and define

$$\Gamma_k = \sum_{i=1}^{k} E_i \qquad \boldsymbol{X}_n = c(\alpha) \sum_{k=1}^{n} \Gamma_k^{-1/\alpha}\, \boldsymbol{V}_k \tag{3}$$

where

$$c(\alpha) = \left(\frac{\kappa}{\Lambda(\mathbb{S}^{d-1})}\right)^{-\frac{1}{\alpha}} \qquad \& \qquad \kappa = \begin{cases} \frac{\Gamma(2-\alpha)\cos(\frac{\pi\alpha}{2})}{1-\alpha} & \alpha \neq 1 \\ \frac{\pi}{2} & \alpha = 1. \end{cases}.$$

- For $0 < \alpha < 1$, $\boldsymbol{X}_n$ converges in total variation to a stable vector $\boldsymbol{X}$ with spectral measure Λ and stability index α.
- For $1 \leq \alpha < 2$, whenever

$$\int_{\mathbb{S}^{d-1}} s\Lambda(ds) = 0, \tag{4}$$

the random vector $\boldsymbol{X}_n$ converges in total variation to a stable vector $\boldsymbol{X}$ with spectral measure Λ and stability index α.

Whenever there is convergence to $\boldsymbol{X}$, we write

$$\boldsymbol{X} = c(\alpha) \sum_{k=1}^{\infty} \Gamma_k^{-1/\alpha}\, \boldsymbol{V}_k.$$

Theorem 1 indicates that sampling a multivariate stable vector can be done by computing the truncated series as in equation (3). Since the vector $\boldsymbol{X}$ resulting from the series has a zero shift vector $\boldsymbol{\mu}^0 = \boldsymbol{0}$ V. Bentkus, A. Juozulynas, and V. Paulauskas [23, Remark 1], we simply add $\boldsymbol{\mu}^0$ after sampling $\boldsymbol{X}_n$ using the LePage series, when needed. The analysis of the truncation error is deferred to subsection A.1 in the Appendix.

In the following section, we present algorithms to sample according to well-known spectral measures, namely: Isotropic, Sub-Gaussian, Discrete, and Mixed spectral measures.

4.2. Common spectral measures

4.2.1. *Discrete spectral measures*

In the case of a discrete spectral measure with M mass points, we store the point masses in an array $[\boldsymbol{v}_1, \boldsymbol{v}_2, \cdots, \boldsymbol{v}_M]$ and draw randomly with the probability of each vector $\boldsymbol{v}_i$ being

$$\Pr(\boldsymbol{v}_i) = \frac{\Lambda(\boldsymbol{v}_i)}{\sum_{\ell=1}^{M} \Lambda(\boldsymbol{v}_\ell)}.$$

4.2.2. *Isotropic spectral measures*

For an isotropic spectral measure, we sample the vectors $\{\boldsymbol{V}_k\}_{1 \leq k \leq n}$ uniformly on $\mathbb{S}^{d-1}$. To this end, several algorithms are available and we refer the reader to S. Schnabel and W.

Janke [24] where various such algorithms are discussed. In our computations, we adopt the method presented in M. E. Muller [25] due to its simplicity and computational efficiency. Our package sets the spectral measure of the sphere to 1, and then scales the truncated LePage series by an appropriate factor q to adjust for the desired scale γ given by the user: When the spectral measure of the sphere is 1, it can be verified that the characteristic function is given by

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = \exp\left\{ -\frac{\Gamma\left(\frac{d}{2}\right)}{\sqrt{\pi}} \frac{\Gamma\left(\frac{\alpha+1}{2}\right)}{\Gamma\left(\frac{\alpha+d}{2}\right)} \parallel \boldsymbol{t} \parallel^{\alpha} \right\}.$$

To achieve a scale γ, we therefore multiply the truncated series by

$$q = \gamma \left[\frac{\Gamma\left(\frac{d}{2}\right)}{\sqrt{\pi}} \frac{\Gamma\left(\frac{\alpha+1}{2}\right)}{\Gamma\left(\frac{\alpha+d}{2}\right)} \right]^{-\frac{1}{\alpha}}.$$

4.2.3. *Sub-Gaussian (elliptical) spectral measures*

A sub-Gaussian random vector is characterized by a positive-definite $d \times d$ shape matrix Σ. If $\boldsymbol{X}$ is a unit-scale isotropic multivariate stable random vector with stability index α then $\boldsymbol{Y} = \Sigma^{\frac{1}{2}}\boldsymbol{X}$ is a sub-Gaussian random vector with shape matrix Σ where $\Sigma^{\frac{1}{2}}$ is the Cholesky decomposition of Σ [4].

One way to sample sub-Gaussian random vectors is by using the definition presented in G. Samorodnitsky and M. S. Taqqu [18, Definition 2.5.1]. This is adopted whenever the α-stable vector is purely sub-Gaussian. Whenever the sub-Gaussian spectral measure is an element of a mixture (see section 4.2.4 for mixed spectral measures), then the approach in G. Samorodnitsky and M. S. Taqqu [18, Definition 2.5.1] is no longer suitable. Instead, we devise a new approach that is based on the following result: if $\boldsymbol{X}_n$ is the n-terms LePage series of an isotropic vector $\boldsymbol{X}$ with unit scale, then, by Theorem 1, $\boldsymbol{X}_n$ converges in total variation to $\boldsymbol{X}$, and

$$\boldsymbol{Y}_n = \Sigma^{\frac{1}{2}}\boldsymbol{X}_n = c(\alpha)\Sigma^{\frac{1}{2}} \sum_{k=1}^{n} \Gamma_k^{-\frac{1}{\alpha}} \boldsymbol{U}_k = c(\alpha) \sum_{k=1}^{n} \Gamma_k^{-\frac{1}{\alpha}} \boldsymbol{V}_k \qquad \text{where } \boldsymbol{V}_k = \Sigma^{\frac{1}{2}}\boldsymbol{U}_k, \tag{5}$$

converges in total variation to $\boldsymbol{Y} = \Sigma^{\frac{1}{2}}\boldsymbol{X}$, which is the desired sub-Gaussian random vector. Hence, to sample the vectors $\boldsymbol{V}_k$ for the sub-Gaussian case, the software samples $\boldsymbol{U}_k$ uniformly distributed on the unit sphere and multiplies them by $\Sigma^{\frac{1}{2}}$.

4.2.4. *Mixed spectral measures*

Now consider the scenario where the spectral measure at hand is a mixture of L different spectral measures: $\Lambda_1, \Lambda_2, \cdots, \Lambda_L$. If $w_1, w_2, \cdots, w_L$ are their respective associated weights, the mixed spectral measure Λ_{mix} is given by

$$\Lambda_{\text{mix}} = \sum_{i=1}^{L} w_i \Lambda_i.$$

To sample from this spectral measure, we proceed as follows.

- For each weight w_i we associate its corresponding probability as

$$p_i = \frac{w_i}{\sum_{\ell=1}^{L} w_\ell} \qquad i = 1, \cdots, L.$$

Then, we sort the probabilities in non-decreasing order that we assume –without loss of generality– to be $p_1, \cdots, p_L$ hereafter.

- We draw $U \sim \mathcal{U}[0, 1]$,
  - If $U \leq p_1$ sample from Λ_1,

- else if $U \leq p_1 + p_2$, sample from Λ_2,
- else iterate on the remaining spectral measures.

This algorithm achieves a sampling from the mixture of $\Lambda_1, \cdots, \Lambda_L$ with probabilities $p_1, \cdots, p_L$ respectively.

5. Implementation

Whenever the LePage series is used, the package chooses the number of terms to achieve an MSE less than 0.01 using Theorem 2 in the Appendix in section A.1. This number is capped at 50,000 terms and a warning is issued whenever this cap is reached. After varying the values of α and the total mass of the unit sphere, a suitable value for the mean square error threshold, one that leads to sufficiently accurate results with reasonable consumption of time and memory is 0.01.

5.1. Univariate random number generator

In the univariate case, the software samples $X \sim \mathcal{S}(\alpha, \beta, \gamma, \delta; k)$, and the type 1 parameterization is used by default. A sample call for both parametrizations is shown in the following block of code.

```
import aub_htp as ht

alpha = 1
beta = 0
loc = 0
scale = 1
n = 500000
samples = ht.alpha_stable.rvs(alpha=alpha, beta=beta, loc=loc, scale=scale, size=n)
# For the type 0 parametrization use:
# ht.alpha_stable.with_parameterization("S0")
# samples = ht.alpha_stable.rvs(alpha=alpha, beta=beta, loc=loc, scale=scale, size=n)
```

5.2. Multivariate random number generator

Isotropic α-stable random vector:
An isotropic α-stable random vector with characteristic function

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = e^{-\gamma^{\alpha} \| \boldsymbol{t} \|^{\alpha} + \mathrm{i}\boldsymbol{t}^T \boldsymbol{\mu}^0},$$

can be generated using the AUB-HTP. A sample syntax –commented out– is shown below in the section of the Elliptic α-stable random vector.

Elliptic α-stable random vector:
An elliptic α-stable random vector with shape matrix Σ with the following characteristic function

$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = e^{-(\boldsymbol{t}^T \Sigma \boldsymbol{t})^{\frac{\alpha}{2}} + \mathrm{i}\boldsymbol{t}^T \boldsymbol{\mu}^0},$$

can also be generated using AUB-HTP. We note that the elliptic sampler takes the total mass of the unit sphere as input. If the total mass of the sphere is not specified, the software estimates it using Monte Carlo simulations (with 10^6 samples) as follows:

$$\Lambda(\mathbb{S}^{d-1}) = \mathbb{E}\left[\| \Sigma^{\frac{1}{2}} \boldsymbol{U} \|^{\alpha}\right] \approx \frac{1}{n} \sum_{i=1}^{n} \| \Sigma^{\frac{1}{2}} \boldsymbol{U}_i \|^{\alpha}, \tag{6}$$

where the equality is due to G. Samorodnitsky and M. S. Taqqu [18, proposition 2.5.8], and where $\boldsymbol{U}_1, \boldsymbol{U}_2, \ldots \boldsymbol{U}_n$ are i.i.d. samples according to $\boldsymbol{U} \sim \mathcal{U}(\mathbb{S}^{d-1})$. Equation (6) is justified by the application of the Weak Law of Large Numbers (WLLN) because the random variable $\| \Sigma^{\frac{1}{2}} \boldsymbol{U} \|^{\alpha}$ has a finite second moment. Indeed,

$$\begin{aligned}\mathbb{E}\left[\| \Sigma^{\frac{1}{2}}\boldsymbol{U} \|^{2\alpha}\right] &= \mathbb{E}\left[\left(\boldsymbol{U}^T\Sigma^{\frac{1}{2}T}\Sigma^{\frac{1}{2}}\boldsymbol{U}\right)^{\alpha}\right] \\ &\overset{(a)}{=} \mathbb{E}\left[\left(\boldsymbol{U}^T\Sigma^T\boldsymbol{U}\right)^{\alpha}\right] \\ &\overset{(b)}{=} \mathbb{E}[\| \boldsymbol{U} \|_{\Sigma}^{2\alpha}] \\ &\overset{(c)}{\leq} \lambda_{\max}^{\alpha}(\Sigma)\mathbb{E}[\| \boldsymbol{U} \|^{2\alpha}] = \lambda_{\max}^{\alpha}(\Sigma) < \infty,\end{aligned} \tag{7}$$

where equality (a) is due to the Cholesky decomposition definition and where in inequality (b) we used the Σ-weighted norm $\| \boldsymbol{x} \|_{\Sigma}^2 \triangleq \boldsymbol{x}^T\Sigma\boldsymbol{x}$. Finally, the inequality in (c) is due to the fact that $\| \boldsymbol{x} \|_{\Sigma}^2 \leq \lambda_{\max}(\Sigma) \| \boldsymbol{x} \|^2$ where $\lambda_{\max}(\Sigma)$ is the largest eigenvalue of Σ, equal to the norm of matrix Σ [26].

Finally, we note that the MSE resulting from estimating the total mass of the sphere using equation (6) is given by H. Niederreiter [27, Theorem 1.1].

An example code for both isotropic and elliptic cases is shown as follows.

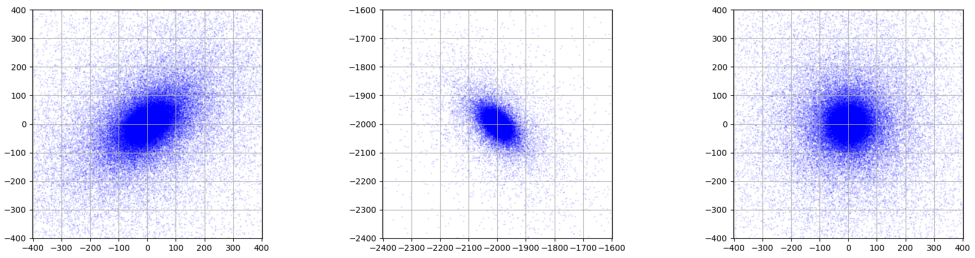
```
import aub_htp as ht
from aub_htp.random import IsotropicSampler, EllipticSampler

alpha = 0.5
gamma = 1
shift = [0, 0]
d = 2
n = 500000
sigma1 = [[2, 0.8], [0.8, 1.5]]
sampler = EllipticSampler(number_of_dimensions=2, alpha=alpha, sigma=sigma1)
samples = ht.multivariate_alpha_stable.rvs(alpha=alpha, spectral_measure_sampler=sampler,
shift=shift, size=n)
# For an Isotropic spectral measure use:
# sampler = IsotropicSampler(number_of_dimensions=d, alpha=alpha, gamma=gamma)
# samples = ht.multivariate_alpha_stable.rvs(alpha=alpha, spectral_measure_sampler=sampler,
shift=shift, size=n)
```

Figures 4a and 4b show the scatter plots of the elliptical α-stable random vector, where we use the following shape matrices

$$\Sigma_1 = \begin{pmatrix} 2 & 0.8 \\ 0.8 & 1.5 \end{pmatrix} \qquad \Sigma_2 = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

Figure 4c shows the scatterplot of an isotropic vector associated with the commented code.



(a) $\alpha = 0.5, \Sigma_1, \boldsymbol{\mu}^{0T} = (0, 0)$ (b) $\alpha = 1.1, \Sigma_2, \boldsymbol{\mu}^{0T} = -(2000, 2000)$ (c) $\alpha = 0.5, \gamma = 1, \boldsymbol{\mu}^{0T} = (0, 0)$

Figure 4. *Scatter plots each containing 5×10^5 points of elliptic contoured and isotropic α-stable distributions; the plots have been truncated to a range of 400 points within the shift vector.*

Discrete spectral measures:
For an α-stable random vector that has a discrete spectral measure with point masses at positions $\boldsymbol{v}_1, \cdots, \boldsymbol{v}_M$ and corresponding weights $w_1, \cdots, w_M$, the corresponding characteristic function is given by

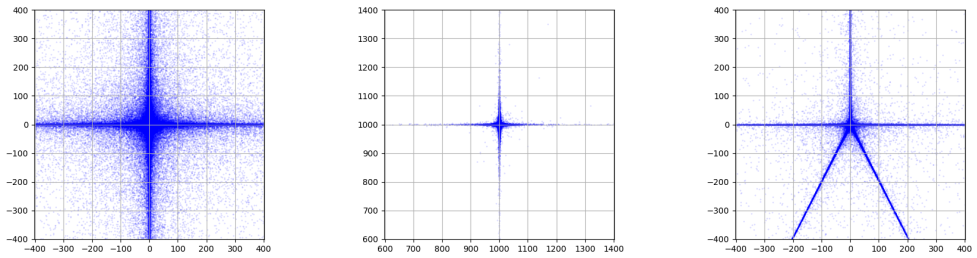
$$\phi_{\boldsymbol{X}}(\boldsymbol{t}) = \exp\left\{ -\sum_{\ell=1}^{M} w_\ell \psi_\alpha(\boldsymbol{t}^T \boldsymbol{v}_\ell) + \mathrm{i}\boldsymbol{t}^T \boldsymbol{\mu}^0 \right\}.$$

Such a vector can be generated as follows:

```
import aub_htp as ht
from aub_htp.random import DiscreteSampler

alpha = 0.2
shift = [0, 0]
d = 2
n = 500000
positions = [[1, 0], [0, 1], [-1, 0], [0, -1]]
weights = [1, 1, 1, 1]
sampler = DiscreteSampler(alpha=alpha, positions=positions, weights=weights)
samples = ht.multivariate_alpha_stable.rvs(alpha=alpha, spectral_measure_sampler=sampler,
shift=shift, size=n)
```

with corresponding scatter plots shown in Figure 5.



(a) Masses $= [1, 1, 1, 1]$, weights $= [(1, 0), (0, 1), (-1, 0), (0, -1)]$, $\alpha = 0.2$, $\boldsymbol{\mu}^{0T} = (0, 0)$.

(b) Masses $= \left[\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}\right]$, weights $= [(1, 0), (0, 1), (-1, 0), (0, -1)]$, $\alpha = 1.2$, $\boldsymbol{\mu}^{0T} = (1000, 1000)$.

(c) Masses $= \left[\frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}\right]$, weights $= [(1, 0), (0, 1), (-1, 0), (\cos(\theta), -\sin(\theta)), (\cos(\pi - \theta), -\sin(\pi - \theta))]$, $\theta = \frac{13\pi}{20}$, $\alpha = 0.1$, $\boldsymbol{\mu}^{0T} = (0, 0)$.

Figure 5. *Scatter plots each containing 5×10^5 points of α-stable distributions with a discrete spectral measure. The plots have been truncated to a range of 400 points within the shift vector.*

Mixed spectral measure:
For a mixed spectral measure, let $\Lambda_1, \cdots, \Lambda_L$ be L spectral measures with associated weights $w_1, \cdots, w_L$, and consider their mixture $\Lambda_{\text{mix}} = \sum_{i=1}^{L} w_i \Lambda_i$. To sample according to this spectral measure, we proceed as follows:

```
import aub_htp as ht
from aub_htp.random import DiscreteSampler, IsotropicSampler, MixedSampler

alpha = 0.3
shift = [1000, 1000]
d = 2
n = 500000
gamma = 1

# Discrete Spectral Measure
positions = [[1, 0], [0, 1], [-1, 0], [0, -1]]
weights = [1, 1, 1, 1]
sampler1 = DiscreteSampler(alpha=alpha, positions=positions, weights=weights)

# Isotropic Spectral Measure
sampler2 = IsotropicSampler(number_of_dimensions=d, alpha=alpha, gamma=gamma)

# Spectral Measures and Their Associated Weights
spectral_measures = [sampler1, sampler2]
measure_weights = [0.5, 0.5]

mixed_sampler = MixedSampler(
    spectral_measures=spectral_measures, weights=measure_weights
)
samples = ht.multivariate_alpha_stable.rvs(
    alpha=alpha, spectral_measure_sampler=mixed_sampler, shift=shift, size=n
)
```

where `spectral_measures` is a list containing the spectral measures that we are mixing, and `measure_weights` contains the associated weight of each measure. The resulting scatter plot is shown in Figure 6a.

Custom spectral measures:
The common spectral measures presented above are built into AUB-HTP. If the user wants to sample from an α-stable random vector whose spectral measure is not built-in, this is made possible using a systematic procedure outlined in the following example:

- Say the user is interested in sampling a multivariate α-stable random vector in dimension $d = 2$ with $\alpha = 0.3, \boldsymbol{\mu}^0 = \mathbf{0}$, and whose spectral measure is uniform over the arcs $[-\frac{\pi}{4}, \frac{\pi}{4}]$ and $[\frac{3\pi}{4}, \frac{5\pi}{4}]$, that is, with probability 0.5, we draw a point uniformly on the arc $[\frac{3\pi}{4}, \frac{5\pi}{4}]$, and with probability 0.5, we draw a point uniformly on the arc $[-\frac{\pi}{4}, \frac{\pi}{4}]$.
- We assign equal probabilities to both arcs and the spectral measure of each arc is half the total measure of the sphere. Generally, the probability of each region is the ratio of the spectral measure of the region to the total mass of the sphere.
- We create a class of the spectral measure sampler that we want by inheriting from the base class `BaseSpectralMeasureSampler`. In the class that the user is creating, the function which samples the vectors $\boldsymbol{V}$ in Theorem 1 should be provided. Moreover, the dimension of the desired alpha-stable vector and the total mass of the unit sphere should be specified. The created spectral measure sampler is then passed as an argument to the `rvs()` method as shown in the following code.

The steps are presented below.

```python
import aub_htp as ht
from aub_htp.random import BaseSpectralMeasureSampler

alpha = 0.3
shift = [0, 0]
d = 2
n = 500000


class ButterflySampler(BaseSpectralMeasureSampler):

    def sample(self, number_of_samples: int, random_state=None):
        p = np.random.rand(number_of_samples)
        theta = np.empty(number_of_samples)

        mask = p <= 0.5
        theta[mask] = np.random.uniform(-np.pi / 4, np.pi / 4, size=mask.sum())
        theta[~mask] = np.random.uniform(3 * np.pi / 4,
                                         5 * np.pi / 4,
                                         size=(~mask).sum())

        x = np.cos(theta)
        y = np.sin(theta)

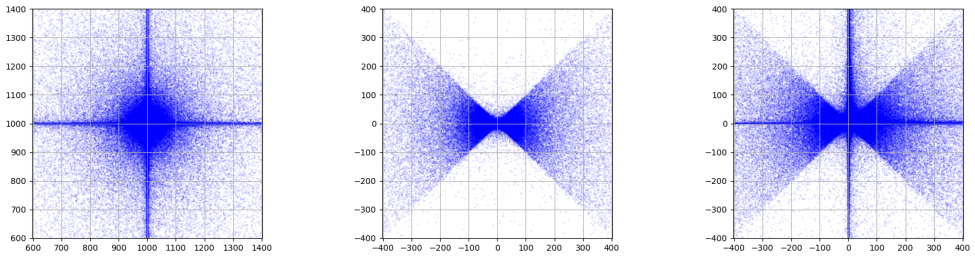
        return np.column_stack((x, y))

    def dimensions(self) -> int:
        return 2

    def mass(self) -> float:
        return 1.0


samples = ht.multivariate_alpha_stable.rvs(
    alpha=alpha,
    spectral_measure_sampler=ButterflySampler(),
    size=n,
    shift=shift)
```

We highlight that the mass of the sphere and the dimension should be specified when implementing the sub-class. The resulting scatter plot is shown in Figure 6b. It is important to emphasize that when $\alpha \geq 1$, the custom spectral measure should satisfy the condition in equation (4).



(a) Mix between isotropic and discrete spectral measures.

(b) A custom spectral measure

(c) Mix between custom and discrete spectral measures.

Figure 6. *Scatter plots showing a custom spectral measure and different mixtures of spectral measures. The plots have been truncated to a range of 400 points within the shift vector.*

One could mix a custom spectral measure with another spectral measure. For example, we mix below equally the spectral measure defined above with the discrete spectral measure with $\text{masses} = [(1,0),(0,1),(-1,0),(0,-1)]$ and $\text{weights} = [1,1,0.25,0.25]$, and the random vector has $\alpha = 0.25$, $\boldsymbol{\mu}^{0T} = (0,0)$.

```
import aub_htp as ht
from aub_htp.random import BaseSpectralMeasureSampler, DiscreteSampler, MixedSampler

alpha = 0.25
shift = [0, 0]
d = 2
n = 1000000


class ButterflySampler(BaseSpectralMeasureSampler):
    ...
    #implementation in previous block of code.


# Discrete Spectral Measure
positions = [[1, 0], [0, 1], [-1, 0], [0, -1]]
weights = [1, 1, 0.25, 0.25]
sampler1 = DiscreteSampler(alpha=alpha, positions=positions, weights=weights)

#Spectral Measures and Their Associated Weights
spectral_measures = [sampler1, ButterflySampler()]
measure_weights = [0.5, 0.5]

mixed_sampler = MixedSampler(spectral_measures=spectral_measures,
                             weights=measure_weights)
samples = ht.multivariate_alpha_stable.rvs(
    alpha=alpha, spectral_measure_sampler=mixed_sampler, shift=shift, size=n)
```

The resulting scatterplot is shown in Figure 6c.

6. Random Variate Generation Validation

To validate the correctness of our random variate generation, we conducted a projection based goodness of fit test. By the stability property, if $\boldsymbol{X}$ is an α-stable random vector in $\mathbb{R}^d$, then $\boldsymbol{\theta}^T\boldsymbol{X}$ is an α-stable random variable $\forall \boldsymbol{\theta} \in \mathbb{R}^d$ G. Samorodnitsky and M. S. Taqqu [18, Theorem 2.1.2]. One way to test the correctness of our generator is therefore to sample random vectors from AUB-HTP, compute the projection of these vectors on some other vectors, and then test the correctness of the resulting projection values which should theoretically have a univariate α-stable distribution. To test the goodness of fit of the resulting projection values, we use the Kolmogorov-Smirnov (KS) test. For details about the formulation of the test, we refer the reader to M. DeGroot and M. Schervish [28, Section 10.6]. We use the implementation of the test provided in the STABLE 5.3 Matlab package [29]. The test returns the D statistic (see M. DeGroot and M. Schervish [28, equation 10.6.2]) and the p-value associated with the test. Smaller values of D and larger values of p indicate a good fit. In all of the following tests, we determined the theoretical values of $\beta, \gamma,$ and δ using G. Samorodnitsky and M. S. Taqqu [18, equation. 2.3.3, equation. 2.3.4, equation. 2.3.5]; we sampled using AUB-HTP vectors with a mean square error less than 0.001; the dimension of the sampled vector was chosen to be $d = 2$; the shift vector was chosen to be $\boldsymbol{\mu}^{0T} = \mathbf{0}$; and 200000 sampled vectors were generated for the test. We tested two spectral measures, Isotropic and Discrete.

6.1. Isotropic Test

The sampled vectors used in this test have unit scale. Since the spectral measure is isotropic, the distribution of the projected values is invariant to the direction of projection. We therefore project along the x-axis and vary only the norm of the projection vector, using $(1, 0)$, $(2, 0)$, $(3, 0)$, $(4, 0)$, and $(5, 0)$. The resulting one-dimensional projections retain the same stability index α as the original vector, with $\beta = 0$, $\delta = 0$, and scale parameters $\gamma = 1, 2, 3, 4, 5$, respectively.

We report the results of the conducted tests for $\alpha = 0.6$ and $\alpha = 1.1$: the results on all the projected vectors gave $D = 0.0021$, $p = 0.344$ for $\alpha = 0.6$, and $D = 0.0013$, $p = 0.9$, for $\alpha = 1.1$. Both results indicate good fit.

6.2. Discrete Test

In this example, the spectral measure has 4 point masses at $(1,0),(0,1),(-1,0),(0,-1)$ with all masses having weight 0.25. We project along the vector $(\cos\theta, \sin\theta)$ and we vary $\theta \in \left\{0, \frac{\pi}{4}, \frac{\pi}{2}, \frac{3\pi}{4}, \pi, \frac{5\pi}{4}, \frac{3\pi}{2}, \frac{7\pi}{4}\right\}$. Tables 3 and 4 show the theoretical values of γ and β that the univariate stable projections should have (all projections have $\delta = 0$), the D statistic, and the p-value. Indeed, we can see that for both values of α and for most projection directions, the D statistic is small and the p-value is relatively large, which indicates a good fit. There are a few cases in which the D statistic is small but the p-value is small as well. In these cases, albeit the value of p is low, the fit is still deemed satisfactory (by virtue of the low value of D). This is due to the known fact that the KS test is highly sensitive at large sample sizes resulting in possibly low p-values even when the observed discrepancy is practically negligible. For a more detailed discussion on the sensitivity of the KS test, we refer the reader to M. DeGroot and M. Schervish [28, Section 10.6].

Table 3. *KS Test Results for* $\alpha = 0.6$

θ	γ	β	D	p
0	0.314980	0	0.001130	0.960380
$\frac{\pi}{4}$	0.707107	0	0.001058	0.978556
$\frac{\pi}{2}$	0.314980	0	0.001915	0.455143
$\frac{3\pi}{4}$	0.707107	0	0.001633	0.660202
π	0.314980	0	0.001130	0.960380
$\frac{5\pi}{4}$	0.707107	0	0.001058	0.978556
$\frac{3\pi}{2}$	0.314980	0	0.001915	0.455143
$\frac{7\pi}{4}$	0.707107	0	0.001633	0.660202

Table 4. *KS Test Results for* $\alpha = 1.1$

θ	γ	β	D	p
0	0.532521	0	0.001474	0.777075
$\frac{\pi}{4}$	0.707107	0	0.001979	0.413518
$\frac{\pi}{2}$	0.532521	0	0.002252	0.262298
$\frac{3\pi}{4}$	0.707107	0	0.001771	0.556735
π	0.532521	0	0.001474	0.777075
$\frac{5\pi}{4}$	0.707107	0	0.001979	0.413518
$\frac{3\pi}{2}$	0.532521	0	0.002252	0.262298
$\frac{7\pi}{4}$	0.707107	0	0.001771	0.556735

7. Conclusion

This paper presented AUB-HTP, a Python package for density computation and random variate generation of α-stable distributions. The package is motivated by the practical difficulty of working with α-stable laws, whose densities generally lack closed-form expressions and whose multivariate simulation requires careful handling of the underlying spectral measure.

AUB-HTP provides scalar density evaluation through a hybrid strategy that combines input standardization, Zolotarev-type integral representations, series formulas, and numerical inversion of characteristic functions. This design improves robustness across parameter regimes and enables systematic comparisons in terms of accuracy and runtime. The validation results show that the package provides reliable scalar density values and identify parameter regimes in which SciPy's reference implementation appears to suffer from numerical inaccuracies.

The package also implements scalar and multivariate random variate generation. For multivariate α-stable vectors, AUB-HTP uses LePage series representations and supports several spectral-measure models, including discrete, isotropic, sub-Gaussian/elliptical, mixed, and custom spectral measures. These features extend the range of α-stable models that can be simulated within the Python ecosystem. The truncation-error analysis further provides guidance on the approximation induced by finite LePage-series representations.

By bringing together density evaluation, flexible simulation, validation, runtime analysis, and extensible implementation tools, AUB-HTP aims to make α-stable modeling more accessible for computational work involving heavy-tailed data. Future developments may include multivariate PDF computation, CDF evaluation, parameter-estimation routines, further numerical acceleration, and application-oriented modules for communications, signal processing, and machine learning.

A. Appendix: Error analysis for the LePage series estimate

In this appendix, we derive an upper bound on the mean square error of the LePage series estimate; we frequently use the following asymptotics of the gamma function from [30]: Let $x, a, b \in \mathbb{R}$, then we have

$$\lim_{x\to\infty} \frac{\Gamma(x+a)}{\Gamma(x+b)\, x^{a-b}} = 1,$$

then $\forall a, b \in \mathbb{R}$, there exists an n_0 such that

$$\frac{\Gamma(k+a)}{\Gamma(k+b)} < 2k^{a-b} \qquad \forall k \geq n_0. \tag{10}$$

A.1. The LePage series truncation error

Theorem 2.

Let $\boldsymbol{X}$ and $\boldsymbol{X}_n$ be as in Theorem 1, and assume in addition that $\parallel \boldsymbol{V}_k \parallel = 1$ for all $k = 1, \cdots, n$. Let n_0 be the same as in (10) then $\forall n \geq \max\{n_0, \frac{2}{\alpha}\}$, $\mathbb{E} \parallel \boldsymbol{X}_n - \boldsymbol{X} \parallel^2 \leq \delta_n$ where

$$\delta_n = \begin{cases} \frac{2\alpha^2}{(1-\alpha)^2} c^2(\alpha) n^{2-\frac{2}{\alpha}} \alpha < 1 \\ \frac{2\alpha}{2-\alpha} c^2(\alpha) n^{1-\frac{2}{\alpha}} \quad \alpha \geq 1. \end{cases}$$

The proof is as follows.

We start with the case $\alpha < 1$. We have

$$\parallel \boldsymbol{X}_n - \boldsymbol{X} \parallel = \parallel c(\alpha) \sum_{k=n+1}^{\infty} \Gamma_k^{-\frac{1}{\alpha}} \boldsymbol{V}_k \parallel \leq c(\alpha) \sum_{k=n+1}^{\infty} \parallel \Gamma_k^{-\frac{1}{\alpha}} \boldsymbol{V}_k \parallel = c(\alpha) \sum_{k=n+1}^{\infty} \Gamma_k^{-\frac{1}{\alpha}},$$

which implies

$$
\begin{aligned}
\mathbb{E}[\parallel \boldsymbol{X}_n - \boldsymbol{X} \parallel^2] &\leq \mathbb{E}\left[\left(c(\alpha)\sum_{k=n+1}^{\infty}\Gamma_k^{-\frac{1}{\alpha}}\right)^2\right] = c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right] + 2\sum_{n<i<j}\mathbb{E}\left[\Gamma_i^{-\frac{1}{\alpha}}\Gamma_j^{-\frac{1}{\alpha}}\right]\right) \\
&\overset{(a)}{\leq} c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right] + 2\sum_{n<i<j}\sqrt{\mathbb{E}\left[\Gamma_i^{-\frac{2}{\alpha}}\right]\mathbb{E}\left[\Gamma_j^{-\frac{2}{\alpha}}\right]}\right) \\
&= c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right] + \left(\sum_{k=n+1}^{\infty}\sqrt{\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right]}\right)^2 - \sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right]\right) \\
&\overset{(b)}{=} c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\sqrt{\frac{\Gamma(k-\frac{2}{\alpha})}{\Gamma(k)}}\right)^2 \\
&\overset{(c)}{<} c^2(\alpha)\left(\sum_{k=n+1}^{\infty}\sqrt{2k^{-\frac{2}{\alpha}}}\right)^2 \\
&= 2c^2(\alpha)\left(\sum_{k=n+1}^{\infty}k^{-\frac{1}{\alpha}}\right)^2 \\
&\overset{(d)}{\leq} \frac{2\alpha^2}{(1-\alpha)^2}c^2(\alpha)n^{2-\frac{2}{\alpha}}
\end{aligned}
\tag{11}
$$

where inequality (a) is due to the Cauchy-Schwarz inequality. The equality in (b) is due to the fact that $\Gamma_k \sim \text{Gamma}(k,1), n \geq \frac{2}{\alpha}$, and $\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right] = \frac{\Gamma(k-\frac{2}{\alpha})}{\Gamma(k)}$, and inequality (c) is due to (10). Inequality (d) is due to the fact that

$$
\sum_{k=n+1}^{\infty}k^{-\frac{1}{\alpha}} \leq \int_n^{\infty}x^{-\frac{1}{\alpha}}dx = \frac{\alpha}{1-\alpha}n^{1-\frac{1}{\alpha}}. \tag{16}
$$

For the case $\alpha \geq 1$, since $\parallel \boldsymbol{V}_k \parallel = 1 \; \forall \; k = 1, \cdots, n$, equation (4) implies $\mathbb{E}[\boldsymbol{V}] = \boldsymbol{0}$. Let $\boldsymbol{V}_j = \left(v_j^1, v_j^2, \cdots, v_j^d\right)$ then we have

$$
\begin{aligned}
\mathbb{E}[\parallel \boldsymbol{X}_n - \boldsymbol{X} \parallel^2] &= c^2(\alpha)\,\mathbb{E}\left[\left\langle\sum_{\ell=n+1}^{\infty}\Gamma_k^{-\frac{1}{\alpha}}\boldsymbol{V}_\ell, \sum_{k=n+1}^{\infty}\Gamma_k^{-\frac{1}{\alpha}}\boldsymbol{V}_k\right\rangle\right] \\
&= c^2(\alpha)\mathbb{E}\left[\sum_{k=n+1}^{\infty}\sum_{\ell=n+1}^{\infty}\sum_{i=1}^{d}\Gamma_k^{-\frac{1}{\alpha}}\Gamma_\ell^{-\frac{1}{\alpha}}v_k^i v_\ell^i\right] \\
&\overset{(a)}{=} c^2(\alpha)\left[\sum_{k=n+1}^{\infty}\sum_{\ell=n+1}^{\infty}\sum_{i=1}^{d}\mathbb{E}\left[\Gamma_k^{-\frac{1}{\alpha}}\Gamma_\ell^{-\frac{1}{\alpha}}\right]\mathbb{E}\left[v_k^i v_\ell^i\right]\right] \\
&\overset{(b)}{=} c^2(\alpha)\sum_{k=n+1}^{\infty}\mathbb{E}\left[\Gamma_k^{-\frac{2}{\alpha}}\right]\mathbb{E}[\parallel \boldsymbol{V}_k \parallel^2] \\
&= c^2(\alpha)\sum_{k=n+1}^{\infty}\frac{\Gamma(k-\frac{2}{\alpha})}{\Gamma(k)} < 2c^2(\alpha)\sum_{k=n+1}^{\infty}k^{-\frac{2}{\alpha}} \\
&\leq \frac{2\alpha}{2-\alpha}c^2(\alpha)n^{1-\frac{2}{\alpha}},
\end{aligned}
\tag{17}
$$

where equality (a) is due to the fact that $\boldsymbol{V}_k \perp\!\!\!\perp \Gamma_k$ and (b) is justified since $v_k^i \perp\!\!\!\perp v_\ell^i, \forall\, k \neq \ell$ and $\mathbb{E}[v_k^i] = 0, \forall\, k$. The rest of the equalities and inequalities follow by similar steps as in the case $\alpha < 1$.

One can clearly see that the bound on the error decays faster for smaller values of α. This indicates that this sampling method can appropriately handle the difficult region of small values of α. In the one dimensional case, the mean square error of the LePage series can be found in A. Janicki and P. Kokoszka [31].

References

[1] M. Nassar, K. Gulati, A. Sujeeth, N. Aghasadeghi, B. Evans, and K. Tinsley, "Mitigating near-field interference in laptop embedded wireless transceivers," in *IEEE International Conference on Acoustics, Speech and Signal Processing*, Las Vegas, NV, Mar. 2008, pp. 1405–1408. doi: 10.1109/ICASSP.2008.4517882.

[2] G. Tsihrintzis, P. Tsakalides, and C. Nikias, "Signal detection in severely heavy-tailed radar clutter," in *Conference Record of The Twenty-Ninth Asilomar Conference on Signals, Systems and Computers*, 1995, pp. 865–869vol.2. doi: 10.1109/ACSSC.1995.540823.

[3] M. E. Crovella, M. S. Taqqu, and A. Bestavros, "Heavy-tailed probability distributions in the World Wide Web," *A practical guide to heavy tails*, vol. 1, pp. 3–26, 1998.

[4] J. P. Nolan, "Multivariate elliptically contoured stable distributions: theory and estimation," *Computational Statistics*, vol. 28, no. 5, pp. 2067–2089, 2013, doi: 10.1007/s00180-013-0396-7.

[5] N. N. Taleb, "Statistical Consequences of Fat Tails: Real World Preasymptotics, Epistemology, and Applications." [Online]. Available: https://arxiv.org/abs/2001.10488

[6] B. V. Gnedenko and A. N. Kolmogorov, *Limit Distributions for Sums of Independent Random Variables*. Reading Massachusetts: Addison-Wesley Publishing Company, 1968.

[7] J. M. Chambers, C. L. Mallows, and B. W. Stuck, "A Method for Simulating Stable Random Variables," *Journal of the American Statistical Association*, vol. 71, no. 354, pp. 340–344, 1976.

[8] J. Royuela-del Val, F. Simmross-Wattenberg, and C. Alberola-López, "libstable : Fast, Parallel, and High-Precision Computation of \alpha-Stable Distributions in R , C/C++ , and MATLAB", *Journal of statistical software*, vol. 78, no. 1, pp. 1–25, 2017.

[9] J. P. Nolan, *Univariate Stable Distributions: Models for Heavy Tailed Data*, 1st Edition 2020. in Springer Series in Operations Research and Financial Engineering. Cham: Springer Nature, 2020.

[10] M. Teimouri, M. Torshizi, A. Mohammadpour, and S. Nadarajah, "alphastable: An R Package for Modelling Multivariate Stable and Mixture of Symmetric Stable Distributions." [Online]. Available: https://arxiv.org/abs/1809.09546

[11] B. J. Swihart and J. P. Nolan, "The R Journal: Multivariate Subgaussian Stable Distributions in R," *The R Journal*, vol. 14, no. 3, pp. 293–302, 2022, doi: 10.32614/RJ-2022-056.

[12] V. M. Zolotarev, *One-Dimensional Stable Distributions*, vol. 65. in Translations of Mathematical Monographs, vol. 65. Providence, RI: American Mathematical Society, 1986.

[13] A. V. Skorohod, "Asymptotic formulas for stable distribution laws," *Selected translations in mathematical statistics and probability*, vol. 1, pp. 157–161, 1961.

[14] H. Pollard, "The Representation of \(e^-x^\lambda\) as a Laplace Integral", *Bulletin of the American Mathematical Society*, vol. 52, no. 10, pp. 908–910, 1946, doi: 10.1090/S0002-9904-1946-08672-3.

[15] H. Bergström, "On Some Expansions of Stable Distribution Functions," *Arkiv för Matematik*, vol. 2, no. 4, pp. 375–378, 1952, doi: 10.1007/BF02591503.

[16] D. Wuertz, M. Maechler, and R. core team members, "stabledist: Stable Distribution Functions," technical report, 2016. [Online]. Available: https://cran.r-project.org/package=stabledist

[17] R. LePage, M. Woodroofe, and J. Zinn, "Convergence to a Stable Distribution Via Order Statistics," *The Annals of probability*, vol. 9, no. 4, pp. 624–632, 1981.

[18] G. Samorodnitsky and M. S. Taqqu, *Stable non-Gaussian random processes : stochastic models with infinite variance*, 1st ed. Boca Raton: Chapman & Hall/CRC, 2000.

[19] A. Wintner, "The Singularities of Cauchy's Distributions," *Duke Mathematical Journal*, vol. 8, pp. 678–681, 1941.

[20] The SciPy Community, "scipy.stats.levy_stable," technical report, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.levy_stable.html

[21] R. Weron, "On the Chambers-Mallows-Stuck method for simulating skewed stable random variables," *Statistics & Probability Letters*, vol. 28, no. 2, pp. 165–171, 1996, doi: https://doi.org/10.1016/0167-7152(95)00113-1.

[22] R. Weron, "Computationally intensive Value at Risk calculations," Berlin, Papers 2004, 32, 2004. [Online]. Available: https://hdl.handle.net/10419/22205

[23] V. Bentkus, A. Juozulynas, and V. Paulauskas, "Lévy-LePage Series Representation of Stable Vectors: Convergence in Variation," *Journal of theoretical probability*, vol. 14, no. 4, pp. 949–978, 2001.

[24] S. Schnabel and W. Janke, "A simple algorithm for uniform sampling on the surface of a hypersphere," *arXiv.org*, 2022.

[25] M. E. Muller, "A note on a method for generating points uniformly on n -dimensional spheres," vol. 2. pp. 19–20, 1959.

[26] R. A. Horn and C. R. Johnson, *Matrix analysis*. Cambridge: Cambridge University Press, 1985.

[27] H. Niederreiter, *Random number generation and quasi-Monte Carlo methods*. in CBMS-NSF Regional Conference Series in Applied Mathematics. Philadelphia, Pa: Society for Industrial, 1992.

[28] M. DeGroot and M. Schervish, *Probability and Statistics*. in Pearson Modern Classics for Advanced Statistics Series. Pearson Education, Incorporated, 2018. [Online]. Available: https://books.google.com.lb/books?id=poCatAEACAAJ

[29] J. Nolan, "Robust Analysis Inc.." [Online]. Available: http://www.robustanalysis.com/

[30] "NIST Digital Library of Mathematical Functions." [Online]. Available: https://dlmf.nist.gov/

[31] A. Janicki and P. Kokoszka, "Computer investigation of the Rate of Convergence of Lepage Type Series to \alpha-Stable Random Variables", *Statistics (Berlin, DDR)*, vol. 23, no. 4, pp. 365–373, 1992.